

MMKV 库简介及使用指南

MMKV 库简介

MMKV 是腾讯开源的高性能键值对存储库，基于 mmap 内存映射技术，替代 Android 原生的 SharedPreferences，具有**速度快、内存占用低、支持多进程、易用性高**等优势。

核心优势

1. 性能：mmap 直接映射文件到内存，读写无需频繁 I/O，比 SharedPreferences 快数倍；
2. 多进程：原生支持多进程读写，无需额外处理；
3. 易用性：API 设计简洁，接近 SharedPreferences；
4. 稳定性：自动处理数据加密、数据修复，避免 SP 的 xml 解析异常问题。

集成步骤（Android）

1. 添加依赖（build.gradle）

Code block

```
1 dependencies {
2     // 核心库
3     implementation 'com.tencent:mmkv:1.3.0'
4     // 如需加密，添加（可选）
5     // implementation 'com.tencent:mmkv-static:1.3.0'
6 }
```

2. 初始化（Application 中）

在自定义 Application 的 `onCreate` 中初始化 MMKV，建议全局只初始化一次：

Code block

```
1 class MyApp : Application() {
2     override fun onCreate() {
3         super.onCreate()
4         // 初始化 MMKV，默认存储路径：/data/data/包名/files/mmkv/
5         val rootDir = MMKV.initialize(this)
6         // 如需自定义存储路径：
7         // val rootDir = MMKV.initialize(getFilesDir().absolutePath + "/mmkv")
8         Log.d("MMKV", "MMKV 初始化完成，存储路径：$rootDir")
9     }
}
```

```
10 }
```

记得在 `AndroidManifest.xml` 中声明自定义 Application:

Code block

```
1 <application
2     android:name=".MyApp"
3     ...>
4 </application>
```

核心 API 使用

MMKV 的核心操作围绕 `MMKV` 实例展开，默认使用「全局实例」，也可创建「自定义实例」（多业务隔离）。

1. 获取 MMKV 实例

Code block

```
1 // 全局默认实例（推荐，大部分场景使用）
2 val mmkv = MMKV.defaultMMKV()
3
4 // 自定义实例（按业务隔离，如 "user_info" 模块）
5 val userMMKV = MMKV.mmkvWithID("user_info", MMKV.MULTI_PROCESS_MODE)
6 // MULTI_PROCESS_MODE: 多进程模式；SINGLE_PROCESS_MODE: 单进程（默认）
```

2. 写入数据

支持常用数据类型：Boolean、Int、Long、Float、Double、String、Set<String>、ByteArray。

Code block

```
1 // 写入基本类型
2 mmkv.putBoolean("is_login", true)
3 mmkv.putInt("user_id", 1001)
4 mmkv.putLong("timestamp", System.currentTimeMillis())
5 mmkv.putString("username", "张三")
6
7 // 写入集合
8 val tags = setOf("Android", "MMKV", "Kotlin")
9 mmkv.putStringSet("tags", tags)
10
11 // 写入字节数组
12 val bytes = "test".toByteArray()
13 mmkv.putBytes("byte_data", bytes)
```

```
14
15 // 【可选】立即刷盘（MMKV 默认异步刷盘，如需同步可调用）
16 mmkv.sync() // 同步刷盘（阻塞）
17 // mmkv.async() // 异步刷盘（默认）
```

3. 读取数据

读取时需指定「默认值」（避免空值）：

Code block

```
1 // 读取基本类型
2 val isLogin = mmkv.getBoolean("is_login", false)
3 val userId = mmkv.getInt("user_id", -1)
4 val username = mmkv.getString("username", "")
5
6 // 读取集合
7 val tags = mmkv.getStringSet("tags", emptySet())
8
9 // 读取字节数组
10 val bytes = mmkv.getBytes("byte_data", ByteArray(0))
```

4. 删除/清空数据

Code block

```
1 // 删除单个键
2 mmkv.removeKey("user_id")
3
4 // 删除多个键
5 mmkv.removeKeys(arrayOf("username", "tags"))
6
7 // 清空所有数据
8 mmkv.clear()
9 // clear(true): 清空并删除文件; clear(false): 仅清空内存（默认）
```

5. 其他常用操作

Code block

```
1 // 判断是否包含某个键
2 val hasKey = mmkv.containsKey("is_login")
3
4 // 获取所有键
5 val allKeys = mmkv.allKeys()
6
```

```
7 // 加密 (初始化时设置密钥)
8 val encryptMMKV = MMKV.mmkvWithID("encrypt_data", MMKV.SINGLE_PROCESS_MODE,
  "123456")
```

注意事项

- 1. 数据类型一致性：写入和读取的类型必须一致（如 putInt 对应 getInt，避免类型转换异常）；
- 2. 多进程场景：创建实例时指定 `MMKV.MULTI_PROCESS_MODE`，无需额外加锁；
- 3. 加密：密钥建议使用 16/24/32 字节（AES 加密），避免硬编码；
- 4. 兼容性：支持 Android API 16+，覆盖绝大多数设备；
- 5. 数据迁移：若从 SharedPreferences 迁移，可遍历 SP 的键值对，批量写入 MMKV。

对比 SharedPreferences

特性	MMKV	SharedPreferences
性能	极快（mmap）	较慢（xml 读写）
多进程	原生支持	需额外处理（易出错）
内存占用	低	高（全量加载 xml）
异常处理	自动修复	易出现 xml 解析异常

总结

MMKV 是 SharedPreferences 的「平替+增强」，API 简洁、性能优异，只需几步集成即可替代原生存储，适合存储轻量级键值对数据（如用户配置、状态标识、缓存信息等）。核心使用流程：**初始化 → 获取实例 → 读写删**，几乎无学习成本，是 Android 端键值存储的首选方案。

（注：文档部分内容可能由 AI 生成）